

Performance Test Plan & Scenarios

1. Executive Summary

This document outlines the performance testing strategy, test flows, and execution profiles for the Demoblaze application.

The framework is designed using **k6** (TypeScript) and validates both the functional correctness and the performance stability of the application under various traffic conditions. It is built to be easily readable by non-technical stakeholders while providing deep architectural hooks for developers.

2. Test Flows & Cases

Flow 1: Homepage Load (homepage.spec.ts)

Description: Simulates a user navigating to the main Demoblaze homepage. This is the most common user journey and is critical for first impressions.

- 1 User sends an HTTP GET request to the homepage URL.
- 2 User pauses for 1 second (simulated "think time" reading the page).

Functional Validations (Pass/Fail):

- Verify the server responds with a 200 OK success status.
- Verify the page content successfully rendered (by searching for the "PRODUCT STORE" text).

Performance Validations (Per-Request):

- Total Duration: Must be under 500ms.
- Waiting Time (TTFB): Time to first byte must be under 200ms.
- Data Transfer: Sending data must take <20ms, and receiving must take <100ms.

Flow 2: Mixed Traffic Simulation (home_page_login_mix.spec.ts)

Description: A realistic, complex scenario simulating concurrent users performing different actions simultaneously. The framework splits the total virtual user count into two distinct groups.

Group A: Browsing (70% of total traffic)

- Simulates standard users viewing the homepage.
- Verifies functional success (200 OK status and correct HTML rendering).

Group B: Login API (30% of total traffic)

- Simulates users attempting to authenticate.
- Sends an HTTP POST request directly to the backend Authentication API (<https://api.demoblaze.com/login>) with JSON credentials.
- Verifies the API successfully processes the request (200 OK) and returns a valid response payload (token or standard error message).

3. Workload Profiles (Execution Strategies)

The framework executes the test flows using predefined "Workloads". A workload dictates how the virtual users behave over time.

Smoke Tests:

- smoke: Generic low-traffic test (fallback scenario, max 5 VUs for 15s).
- smokeLow: Up to 15 VUs for 15s.
- smokeMed: Up to 250 VUs for 15s.
- smokeHigh: Up to 500 VUs for 15s.
- smokeMixedTraffic: Simulates mixed actions simultaneously. It splits traffic into 70% Homepage Browsing and 30% Login API.
- **Goal:** To verify that the test scripts, API endpoints, and test environments are functioning correctly and can handle moderate baselines.

Load Tests:

- load / smokeLoad: Simulates 1,000 concurrent users hitting the application within a 15-second window.
- smokeLoadRampUsers: Ramps up to 1,000 users over a few seconds, holds, and ramps down.
- smokeLoadConstantArrival: Maintains a constant arrival rate of 1,000 requests per second.
- smokeLoadConstantVUs: Maintains exactly 1,000 concurrent VUs dynamically.
- **Goal:** To measure application stability, identify bottlenecks, and observe how the infrastructure scales under expected peak traffic.

4. Success Criteria (Global Thresholds)

Regardless of the flow or workload, the framework enforces strict global Service Level Agreements (SLAs). If the application breaches any of these thresholds under load, the CI/CD pipeline will automatically fail:

Error Rate & Assertions:

- Less than 1% of all HTTP requests are allowed to fail.
- Minimum 95% pass rate for all logical test assertions (content validation, etc.).

Response Time:

- 99% of all requests must complete in under 300ms (which also guarantees p95 < 500ms).
- The average response time across the entire test must remain under 300ms.

Network & Backend Bottlenecks:

- Waiting (TTFB): Time to first byte must be under 200ms for 95% of requests.
- TLS Handshaking: SSL negotiation must be under 100ms for 95% of requests.
- Server Connection: Initial server TCP connections must be established in under 50ms for 95% of requests.
- Sending / Receiving: Sending payloads must take <20ms, and receiving payloads must take <100ms for 95% of requests.
- Blocked TCP Slots: Time blocked waiting for an open TCP connection must be <50ms for 95% of requests.

Execution Engine:

- Iteration (scenario loop) duration must be under 2s for 95% of VUs to guarantee pacing.